

DL Data Structures & Frameworks



Parivesh Priye



INTERVIEW
KICKSTART

Introduction – Parivesh Priye

Applied Scientist, Amazon

(Amazon Search)

Columbia University

Optimize Your Experience

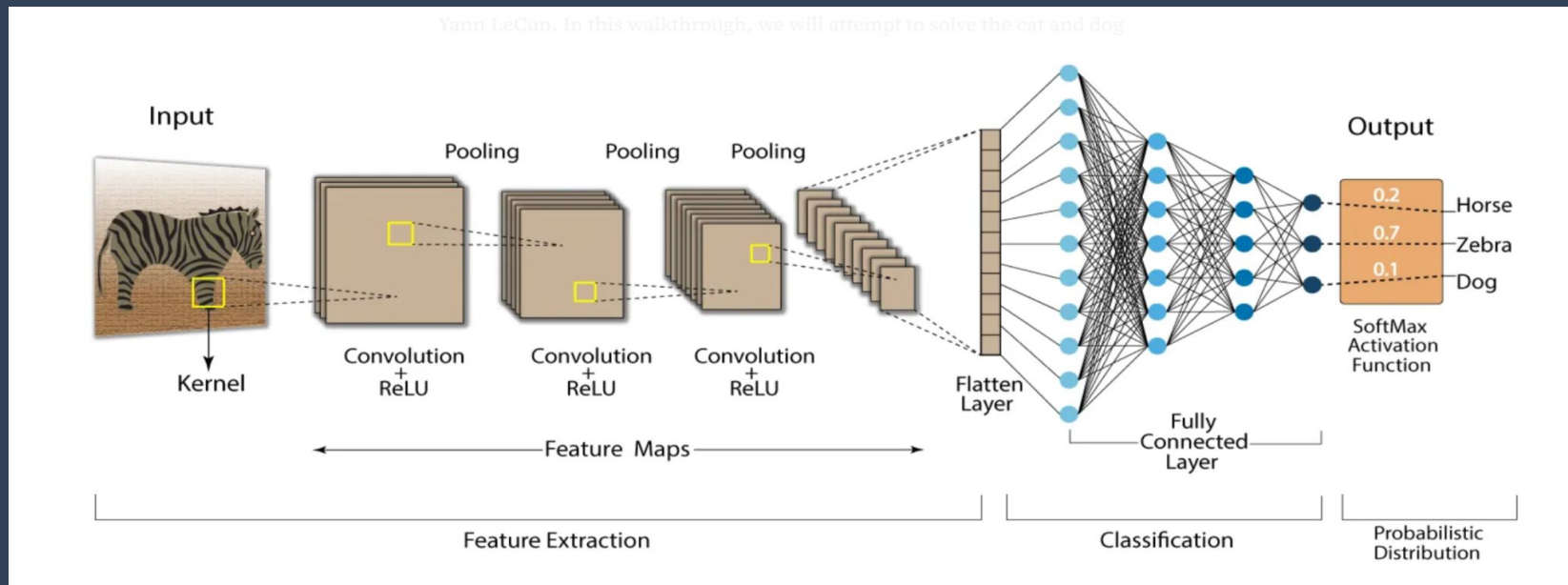
1. Interact with your instructors via Sunday live class.
2. Don't be shy to speak up and get clarifications !
3. Solve your assignments, MCQs and other assessments and get feedback (Thursday review session)
4. Use your resources! It's your experience – what you put in is what you'll get out.

Agenda

- **Topics we will cover today**
 - Introduction to tensors - why are they important
 - General flow of a DL problem
 - Tensor examples from Vision, NLP, Video
 - Tensor manipulations and operations
 - Automatic differentiation using pytorch
 - JIT / JAX

Introduction to Tensors

A network showing tensor formations and image classification



Source:

<https://larbifarihi.medium.com/classifying-cats-and-dogs-using-convolutional-neural-networks-cnns-ce6bc7bad64d>

Learning Objective

- Understanding tensors and tensor operations
- Exploring tensor manipulations and visualizations by performing tensor operations
- Getting a visualization of tensors with real world data like text, image etc.
- Understanding the inner working of Pytorch by understanding Autograd

What are Tensors?

- Tensors are mathematical objects used in deep learning to represent multi-dimensional arrays of numbers.
- For example, in image processing, a tensor can represent a grayscale image, where each element is a pixel value.
- Tensors are often used to create word embeddings in NLP tasks. They are vector representations of words that capture semantic relationships and contextual information.

Hook

Did you know that deep learning tensors can have an astronomical number of dimensions? For instance, the famous Google Inception-v3 network has a tensor with the shape (1, 2048, 8, 8), which means it has 1 channel, 2048 feature maps, and each feature map has a size of 8x8. This enormous number of dimensions allows deep learning models to capture intricate patterns and representations, enabling them to tackle complex tasks such as image recognition with remarkable accuracy.

In the following chapters we will learn what a tensor is and how we can understand the dimensions,

Example of an image tensor:

```
Tensor shape: [3, 224, 224]
```

```
Example values:
```

```
[[[R, R, R, ..., R],  
  [R, R, R, ..., R],  
  ...  
  [R, R, R, ..., R]],
```

```
[[G, G, G, ..., G],  
 [G, G, G, ..., G],  
 ...  
 [G, G, G, ..., G]],
```

```
[[B, B, B, ..., B],  
 [B, B, B, ..., B],  
 ...  
 [B, B, B, ..., B]]]
```

In this example, the tensor has three channels (R, G, and B), and each channel consists of a 224x224 grid of pixel values. The RGB values (R, G, B) represent the intensity of each color channel at each pixel location.

Example of tensor from text

Tensor shape: [7, 5]

Example values:

```
[[0.2, 0.4, -0.1, 0.8, 0.5],  
 [0.7, -0.3, 0.6, -0.2, 0.9],  
 [0.1, 0.2, 0.3, 0.4, 0.5],  
 [0.6, 0.5, 0.4, 0.3, 0.2],  
 [0.8, -0.5, -0.3, 0.2, -0.1],  
 [0.9, 0.1, 0.7, -0.4, 0.3],  
 [0.4, 0.3, 0.2, 0.1, 0.0]]
```

In this example, the tensor represents a sentence with seven words, and each word is represented by a vector of dimension 5. The values in the tensor represent the word embeddings, which capture the semantic meaning or contextual information of each word in the sentence.

Creating Tensors in PyTorch

`torch.rand(2, 3, 4)` creates a tensor with dimensions $2 \times 3 \times 4$, filled with random values between 0 and 1 from a uniform distribution.

Result:

```
import torch

# Create a 3-dimensional tensor with random values from a uniform distribution
random_tensor = torch.rand(2, 3, 4)
print(random_tensor)

tensor([[[[0.7396, 0.9041, 0.6707, 0.0388],
          [0.3166, 0.2164, 0.3055, 0.5180],
          [0.7295, 0.1111, 0.8134, 0.4052]],
        [[0.1256, 0.5521, 0.3843, 0.2436],
          [0.9633, 0.3447, 0.7598, 0.0785],
          [0.6404, 0.1337, 0.2478, 0.5415]]]])
```

Other important methods to create tensors

- `torch.zeros(a, b)`
This pytorch method creates an, a x b shaped tensor with zeros
- `torch.ones(a, b)`
This pytorch method creates an, a x b shaped tensor with ones.

Question: How would you create an identity tensor using `torch.ones` and/or `torch.zeros`?
An identity tensor can be thought of as a square matrix.

Example:

```
tensor([[1., 0., 0., 0.],  
        [0., 1., 0., 0.],  
        [0., 0., 1., 0.],  
        [0., 0., 0., 1.]])
```

Identity Tensor

```
def identity_matrix_with_diag(n):  
    identity_tensor = torch.ones(n, n) #  
    # In the return statement, torch.diag converts the identity_tensor to one dimension  
    # Using torch.diag again creates a 2D tensor from the 1D tensor  
    return torch.diag(torch.diag(identity_tensor))  
  
def identity_matrix_with_sum(n):  
    zeros_tensor = torch.zeros(n, n)  
    identity_tensor = torch.diag(torch.ones(n)) # torch.diag changes 1D to 2D tensor  
    identity_matrix = zeros_tensor + identity_tensor # Element wise addition of two tensors  
    return identity_matrix
```

Basic Mathematical operation on tensors

- Addition: Adds corresponding elements of tensors together, allowing for combining information or increasing values.
- Subtraction: Subtracts corresponding elements of tensors, useful for computing differences or detecting changes.
- Multiplication: Performs element-wise multiplication of tensors, enabling scaling or emphasizing certain features.
- Division: Divides corresponding elements of tensors, useful for normalization or finding relative proportions.

The next slide details example code for the operations.

Basic mathematical operations on tensors

```
import torch

# Create a base tensor
base_tensor = torch.tensor([[1, 2], [3, 4]])

# Create another tensor
second_tensor = torch.tensor([[2, 2], [1, 1]])

# Addition
addition_tensor = base_tensor + second_tensor
print("Addition:\n", addition_tensor)

# Subtraction
subtraction_tensor = base_tensor - second_tensor
print("Subtraction:\n", subtraction_tensor)

# Multiplication
multiplication_tensor = base_tensor * second_tensor
print("Multiplication:\n", multiplication_tensor)

# Division
division_tensor = base_tensor / second_tensor
print("Division:\n", division_tensor)
```

Addition:

```
tensor([[3, 4],
        [4, 5]])
```

Subtraction:

```
tensor([[ -1,  0],
        [ 2,  3]])
```

Multiplication:

```
tensor([[ 2,  4],
        [ 3,  4]])
```

Division:

```
tensor([[0.5000, 1.0000],
        [3.0000, 4.0000]])
```


Tensor indexing and slicing

Let's create a tensor for understanding:

```
import torch

# Create a tensor with size (2, 3, 3)
tensor = torch.tensor([
    [[1, 2, 3], [4, 5, 6], [7, 8, 9]],
    [[10, 11, 12], [13, 14, 15], [16, 17, 18]]
])

print("Original Tensor:")
print(tensor)
print("\nTensor Shape:", tensor.shape)
```

```
Original Tensor:
tensor([[[ 1,  2,  3],
         [ 4,  5,  6],
         [ 7,  8,  9]],
        [[10, 11, 12],
         [13, 14, 15],
         [16, 17, 18]]])
```

```
Tensor Shape: torch.Size([2, 3, 3])
```

This is a 2 x 3 x 3 dimensional tensor.

Let's consider the 0th dimension which is 2 and call it batch dimension

The 1st dimension which is 3 as the row dimension

The 2nd dimension which is 3 as the column dimension

Please remember the the dimensions types will depend on the data that we are working on and this is a naming convention for simplicity.

Tensor indexing and slicing

Selection along the row dimension:

```
# Row Slice (Slice of the second row)
row_slice = tensor[:, 1, :]
print("\nRow Slice:")
print(row_slice)
print("Row Slice Shape:", row_slice.shape)
```

```
Row Slice:
tensor([[ 4,  5,  6],
        [13, 14, 15]])
Row Slice Shape: torch.Size([2, 3])
```

In this example we select [all batches, 2nd row from each batch, all columns].

Tensor indexing and slicing

Selection along the column dimension:

```
# Column Slice (Slice of the third column)
column_slice = tensor[:, :, 2]
print("\nColumn Slice:")
print(column_slice)
print("Column Slice Shape:", column_slice.shape)
```

Column Slice:

```
tensor([[ 3,  6,  9],
        [12, 15, 18]])
```

Column Slice Shape: torch.Size([2, 3])

In this example we select [all batches, all rows from each batch, 3rd column(2nd index) from all the batches].

Tensor indexing and slicing

Selection along the both, row and column dimension:

```
# Mixed Slice (Mixed slice of rows and columns)
mixed_slice = tensor[:, 0:2, 1:3]
print("\nMixed Slice:")
print(mixed_slice)
print("Mixed Slice Shape:", mixed_slice.shape)
```

```
Mixed Slice:
tensor([[[ 2,  3],
         [ 5,  6]],

        [[11, 12],
         [14, 15]]])
Mixed Slice Shape: torch.Size([2, 2, 2])
```

- In this example we select [all batches, the first and the second row from all batches, the 2nd and the 3rd column from all the batches and the sliced rows].
- 0:2 indicates selection of rows of index 0 to index 1. Similarly 1:3 indicates selection of columns index 1 till index 2. Basically if the selection is 0:n, we select 0 to n-1 from the dimension specified.

Tensor Broadcasting

- Tensor broadcasting is a mechanism in PyTorch that enables element-wise operations between tensors of different shapes.
- It automatically adjusts the shapes of tensors to make them compatible for element-wise operations.
- Dimensions are expanded (replicated) to match the shape of the larger tensor.

Tensor Broadcasting Examples

```
tensor1 = torch.tensor([[1, 2, 3], [4, 5, 6]])  
tensor2 = torch.tensor([10, 20, 30])  
result = tensor1 + tensor2  
print("Broadcast addition:\n", result)
```

```
Broadcast addition:  
tensor([[11, 22, 33],  
        [14, 25, 36]])
```

In this addition example, tensor2 is broadcasted to have the same shape as tensor1 by replicating its values along the new dimensions. It followed the rule of matching the shape of the larger tensor.

Tensor Broadcasting Examples

```
tensor1 = torch.tensor([[1, 2, 3], [4, 5, 6]]) # shape: (2,3)
tensor2 = torch.tensor([[10], [20]]) # shape: (2,1)
result = tensor1 * tensor2
print("Broadcast Multiplication:\n", result)
```

```
Broadcast Multiplication:
tensor([[ 10,  20,  30],
        [ 80, 100, 120]])
```

In this multiplication example, tensor2 is broadcasted to have the same shape as tensor1 by replicating its values along the new dimensions. It followed the rule of matching the shape of the larger tensor.

Tensor reduction

- In PyTorch, tensor reduction refers to the process of reducing the dimensions of a tensor by performing mathematical operations that aggregate or summarize the tensor elements along one or more axes.
- The result is a tensor with fewer dimensions or even a scalar value, depending on the reduction operation applied.

Tensor reduction

- Sum reduction: Computes the sum of all tensor elements or along specific dimensions using the `torch.sum()` function.
- Mean reduction: Calculates the mean (average) value of all tensor elements or along specific dimensions using the `torch.mean()` function.
- Max reduction: Finds the maximum value among all tensor elements or along specific dimensions using the `torch.max()` function.
- Min reduction: Determines the minimum value among all tensor elements or along specific dimensions using the `torch.min()` function.
- Argmax reduction: Returns the indices of the maximum values along a given axis using the `torch.argmax()` function.
- Argmin reduction: Returns the indices of the minimum values along a given axis using the `torch.argmin()` function.

Tensor reduction examples

```
# Create a tensor
tensor = torch.tensor([[1, 2, 3],
                       [4, 5, 6],
                       [7, 8, 9]])

# Tensor Reduction Operations
sum_result = torch.sum(tensor)
mean_result = torch.mean(tensor)
max_result = torch.max(tensor)
min_result = torch.min(tensor)
argmax_result = torch.argmax(tensor)
argmin_result = torch.argmin(tensor)
```

```
Results:

Tensor:
tensor([[1, 2, 3],
        [4, 5, 6],
        [7, 8, 9]])

Sum Result: 45
Mean Result: 5.0
Max Result: 9
Min Result: 1
Argmax Result: 8
Argmin Result: 0
```

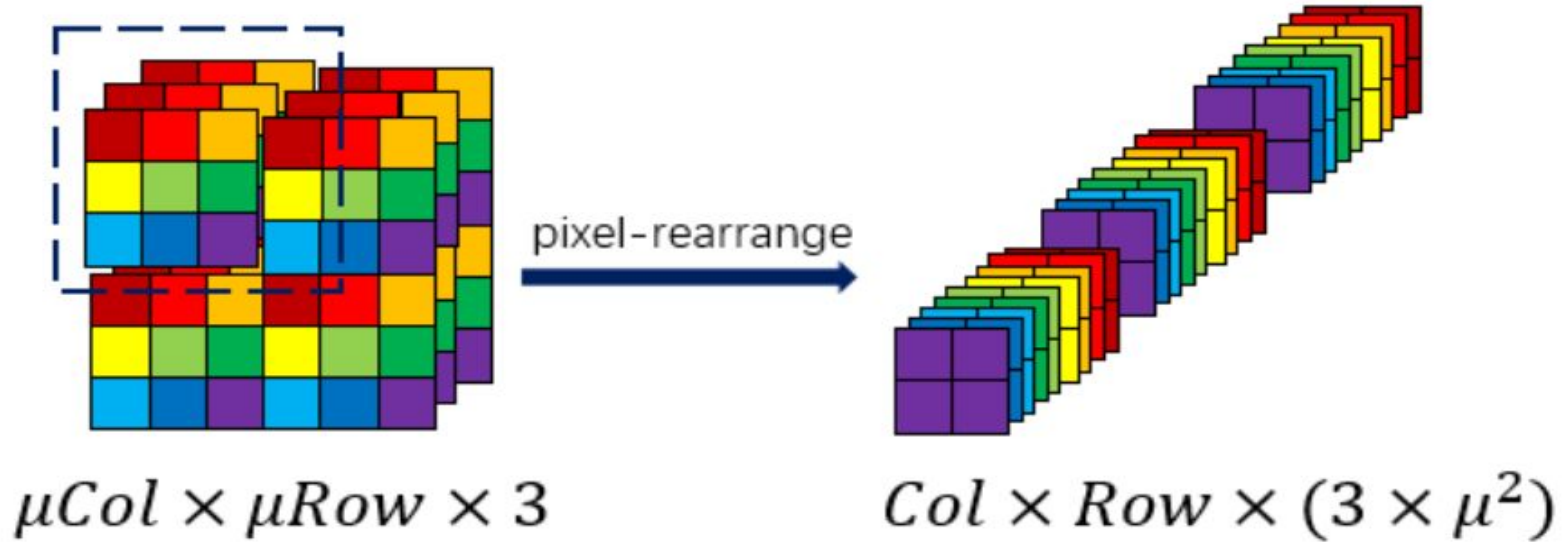
Notebook: [Tensor reduction](#)

Summary

- In this topic we understood the definition of a tensor, defining it and basic idea behind some of the tensor methods like reduction, broadcasting, indexing and slicing.
- In the next chapter we delve into some advanced operations with Tensors and using these operations for different uses.

Tensor Reshaping and Visualization

Hook



[Source](#)

Tensor reshaping

- In the previous slides we understood how to select specific parts of the tensor using batch, row and column dimension selection.
- In this topic we deep dive into reshaping of the tensors, combining and splitting them across different dimensions.
- Two of the most used methods for tensor reshaping are `view()` and `reshape()`

Question: `view()` and `reshape()` perform the similar action of reshaping a tensor. Why do you think there are two methods for the similar objective?

Tensor reshaping with view()

- This function returns a view of the original tensor with the specified shape.
- `torch.view()` operates on contiguous tensors only.
- If the original tensor is contiguous (meaning the memory is laid out as a single block), `view()` will return a new tensor with the desired shape.
- If the original tensor is non-contiguous, `view()` will raise an error.
- It is recommended to use `view()` when working with contiguous tensors.

Tensor reshaping with reshape()

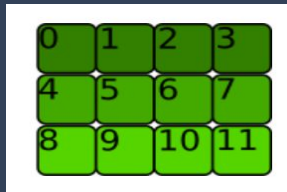
- This function returns a new tensor with the desired shape
- Operates on contiguous and non-contiguous tensors.
- If the desired shape is compatible with the number of elements in the original tensor, reshape() will return the reshaped tensor.
- If the desired shape is incompatible with the number of elements, it will raise an error.

Contiguous() in pytorch

A two-dimensional array is contiguous if its items are laid out in memory next to each other just like below:

Create tensor: `t = torch.tensor([[0, 1, 2, 3], [4, 5, 6, 7], [8, 9, 10, 11]])`

It is laid out in the memory like:



0	1	2	3
4	5	6	7
8	9	10	11

The memory allocation is C contiguous if the rows are stored next to each other like:



0	1	2	3	4	5	6	7	8	9	10	11
---	---	---	---	---	---	---	---	---	---	----	----

To check if a tensor is contiguous Pytorch's inbuilt function can be used: `t.is_contiguous()`

This return True if tensor is contiguous and False if otherwise.

Examples of view and reshape

```
# Create a contiguous 2D tensor
tensor = torch.tensor([[1, 2, 3],
                       [4, 5, 6]])

# Reshape the tensor using view
reshaped_tensor = tensor.view(3, 2)

print("Original Tensor:")
print(tensor)
print("\nReshaped Tensor:")
print(reshaped_tensor)
```

Original Tensor:

```
tensor([[1, 2, 3],
        [4, 5, 6]])
```

Reshaped Tensor:

```
tensor([[1, 2],
        [3, 4],
        [5, 6]])
```

Examples of view and reshape

```
import torch

# Create a non-contiguous 2D tensor using slicing
tensor = torch.tensor([[1, 2, 3, 4, 5, 6]])
sliced_tensor = tensor[:, :3]

# Reshape the tensor using reshape
reshaped_tensor = torch.reshape(sliced_tensor, (1, 3))

print("Original Tensor:")
print(sliced_tensor)
print("\nReshaped Tensor:")
print(reshaped_tensor)
```

Original Tensor:
tensor([[1, 2, 3]])

Reshaped Tensor:
tensor([[1, 2, 3]])

The -1 trick in view()

- The -1 trick in view() is a convenient way to infer the size of a particular dimension when reshaping a tensor. By specifying -1 for one dimension, PyTorch automatically calculates the appropriate size based on the total number of elements and the sizes of the other dimensions.

```
import torch

# Create a tensor with size (2, 4)
tensor = torch.tensor([[1, 2, 3, 4],
                       [5, 6, 7, 8]])

# Reshape the tensor using view and -1 trick
reshaped_tensor = tensor.view(-1, 2)

print("Original Tensor:")
print(tensor)
print("\nReshaped Tensor:")
print(reshaped_tensor)
```

Original Tensor:

```
tensor([[1, 2, 3, 4],
        [5, 6, 7, 8]])
```

Reshaped Tensor:

```
tensor([[1, 2],
        [3, 4],
        [5, 6],
        [7, 8]])
```

The -1 trick in view()

- The -1 in the view() function allows PyTorch to calculate the size of the missing dimension by dividing the total number of elements (8) by the known size of the other dimension (2), resulting in a reshaped tensor of size (4, 2).
- By specifying -1 for one dimension, you let PyTorch calculate the correct size based on the other dimensions and the total number of elements. This saves you from manually computing the size and ensures that the reshaping is done correctly.

Question: Consider a tensor of shape (2,3,4). We try to reshape with view(1,4,-1). What would be the inferred value of -1 ?

Squeeze function

- The squeeze function in PyTorch removes dimensions with size 1 from a tensor.
- It returns a new tensor with the same data but with dimensions of size 1 removed.
- If no arguments are provided, squeeze removes all dimensions of size 1 from the input tensor.
- If the dim argument is specified, squeeze only removes the dimensions of size 1 along that particular dimension.
- When we apply squeeze to an example tensor of shape (2,1,3), the dimensions of size 1 along the second axis are removed, resulting in a squeezed tensor with a shape of (2, 3).

Unsqueeze function

- The unsqueeze function in PyTorch adds dimensions of size 1 to a tensor.
- It returns a new tensor with additional dimensions of size 1 at the specified positions.
- The dim argument specifies the position(s) at which the new dimensions are added.
- The dimensions added by unsqueeze are placed before the specified dimension, shifting the existing dimensions to higher indices.
- If we have a tensor of shape (2,3) and we call `torch.unsqueeze(dim =0)` then our result will be a new tensor of shape (1,2,3).

Transpose

The `torch.transpose` function in PyTorch is used to permute the dimensions of a tensor. It returns a new tensor with the dimensions rearranged according to the specified permutation.

```
import torch

# Example tensor
x = torch.tensor([[1, 2, 3], [4, 5, 6]])
print("Original tensor:")
print(x)
print("Shape of original tensor:", x.shape)

# Transpose the tensor
y = torch.transpose(x, 0, 1)
print("Transposed tensor:")
print(y)
print("Shape of transposed tensor:", y.shape)
```

```
Original tensor:
tensor([[1, 2, 3],
        [4, 5, 6]])
Shape of original tensor: torch.Size([2, 3])
Transposed tensor:
tensor([[1, 4],
        [2, 5],
        [3, 6]])
Shape of transposed tensor: torch.Size([3, 2])
```


Concatenating tensors

- Concatenates the given sequence of tensors in the given dimension. All tensors must either have the same shape (except in the concatenating dimension) or be empty.
- The `torch.cat()` function is commonly used for tensor concatenation.
- In this example we concatenate along dimension 1

```
import torch

# Create tensors
tensor1 = torch.tensor([[1, 2, 3],
                        [4, 5, 6],
                        [7, 8, 9]])
tensor2 = torch.tensor([[10, 11, 12],
                        [13, 14, 15],
                        [16, 17, 18]])

# Concatenate tensors along dimension 1
concatenated_tensor = torch.cat((tensor1, tensor2), dim=1)

# Display results
print("Tensor 1:")
print(tensor1)
print("\nTensor 2:")
print(tensor2)
print("\nConcatenated Tensor:")
print(concatenated_tensor)
```

```
Tensor 1:
tensor([[1, 2, 3],
        [4, 5, 6],
        [7, 8, 9]])

Tensor 2:
tensor([[10, 11, 12],
        [13, 14, 15],
        [16, 17, 18]])

Concatenated Tensor:
tensor([[ 1,  2,  3, 10, 11, 12],
        [ 4,  5,  6, 13, 14, 15],
        [ 7,  8,  9, 16, 17, 18]])
```

Splitting tensors

- Splits the tensor into chunks. Each chunk is a view of the original tensor..
- The `torch.split()` function is commonly used for tensor splitting.
- In this example we split along the dimension 0

```
import torch

# Create a tensor
tensor = torch.tensor([[1, 2, 3],
                       [4, 5, 6],
                       [7, 8, 9],
                       [10, 11, 12]])

# Split the tensor along dimension 0 into two parts
split_tensors = torch.split(tensor, split_size_or_sections=2, dim=0)

# Display results
print("Original Tensor:")
print(tensor)
print("\nSplit Tensors:")
for split_tensor in split_tensors:
    print(split_tensor)
```

Original Tensor:

```
tensor([[ 1,  2,  3],
        [ 4,  5,  6],
        [ 7,  8,  9],
        [10, 11, 12]])
```

Split Tensors:

```
tensor([[1, 2, 3],
        [4, 5, 6]])
tensor([[ 7,  8,  9],
        [10, 11, 12]])
```

Stacking tensors

- Specifying `dim=-1`, we are indicating that we want to stack the tensors along the last dimension.
- In PyTorch, negative values for `dim` are used to index dimensions from the end. When we stack `tensor1` and `tensor2` along `dim=-1`, the resulting `stacked_tensor` will have a new dimension at the end. This means that the elements from both tensors will be placed along this new dimension.
- The reason this result might be confusing is because it introduces a new dimension and changes the shape of the tensor.
- Instead of having a shape of `(3, 3)` for each tensor, the `stacked_tensor` has a shape of `(3, 3, 2)`, indicating that it has two sets of `(3, 3)` elements stacked together along the last dimension.

Stacking tensors

```
import torch

# Create tensors
tensor1 = torch.tensor([[1, 2, 3],
                        [4, 5, 6],
                        [7, 8, 9]])
tensor2 = torch.tensor([[10, 11, 12],
                        [13, 14, 15],
                        [16, 17, 18]])

# Stack tensors along a new dimension
stacked_tensor = torch.stack((tensor1, tensor2), dim=-1)

# Display results
print("Tensor 1:")
print(tensor1)
print("\nTensor 2:")
print(tensor2)
print("\nStacked Tensor:")
print(stacked_tensor)
```

```
Tensor 1:
tensor([[1, 2, 3],
        [4, 5, 6],
        [7, 8, 9]])

Tensor 2:
tensor([[10, 11, 12],
        [13, 14, 15],
        [16, 17, 18]])

Stacked Tensor:
tensor([[[ 1, 10],
         [ 2, 11],
         [ 3, 12]],
        [[ 4, 13],
         [ 5, 14],
         [ 6, 15]],
        [[ 7, 16],
         [ 8, 17],
         [ 9, 18]])])
```

Additional references to understand dimensions

<https://towardsdatascience.com/understanding-dimensions-in-pytorch-6edf9972d3be>

Summary

- In this topic we understood some of the advanced operations done with tensors like view, reshape etc.
- We also understood methods of tensor storage in memory with contiguous and stride.
- In the next chapter we go through Autograd which makes the core of any deep learning framework.

Automatic Differentiation and Autograd

The need for Pytorch Autograd

Gradient Computation:

Autograd allows for the automatic computation of gradients, which are essential for training deep learning models using techniques like backpropagation.

Gradients provide information about how each parameter in the model contributes to the overall loss, allowing for efficient updates during the optimization process.

Flexibility in Model Design:

With autograd, you can define complex and arbitrary computational graphs without having to manually derive and implement the gradients for each operation.

This flexibility enables researchers and practitioners to experiment with various architectures, loss functions, and optimization techniques, accelerating the development and innovation in the field of deep learning.

Efficient Gradient Calculation:

Pytorch's autograd engine efficiently computes gradients using the chain rule of calculus.

It leverages the computational graph to dynamically track the operations performed on tensors and computes the gradients by propagating them through the graph in a backward pass.

This efficient gradient calculation allows for faster training and optimization of deep learning models.

Support for Dynamic Graphs:

Pytorch's dynamic computational graph allows for the creation of dynamic models where the structure of the graph can change during runtime.

This flexibility is particularly beneficial in scenarios where the model needs to handle inputs of varying sizes or lengths, such as in natural language processing tasks or handling sequences of different lengths in recurrent neural networks.

Major concepts of Pytorch Autograd

Automatic Differentiation:

Automatic differentiation is a fundamental technique used in PyTorch for computing gradients of tensors with respect to other tensors. It enables the efficient calculation of gradients, which are crucial for training deep learning models using techniques like backpropagation.

Computational Graph:

PyTorch uses a computational graph to track operations performed on tensors and compute their gradients. The computational graph is a directed acyclic graph (DAG) that represents the sequence of operations applied to tensors.

Tracking Operations:

By default, PyTorch does not track operations on tensors. To enable tracking, tensors need to have their `requires_grad` flag set to `True`. When operations are performed on a tensor with `requires_grad=True`, PyTorch builds a computational graph and tracks the operations for subsequent gradient computation.

Computing Gradients:

The `backward()` function is used to compute gradients of a tensor with respect to a scalar. When `backward()` is called on a tensor, PyTorch traverses the computational graph backward, applying the chain rule to compute the gradients of intermediate tensors.

Example of gradient calculation

```
import torch

# Enable gradient tracking
x = torch.tensor(3.0, requires_grad=True)

# Perform operations on the tensor
y = 2 * x + 1
z = y ** 2

# Compute gradients
z.backward()

# Access the computed gradients
x_grad = x.grad
```

`z.backward()`: This line triggers the backward pass in Pytorch's autograd mechanism. It computes the gradients of `z` with respect to all the tensors that have `requires_grad=True`, in this case, only `x`.

`x_grad = x.grad`: After the backward pass, we can access the gradients of `x` using the `grad` attribute. The computed gradients are stored in `x.grad`.

The value of `x_grad` is `tensor(28.)`. We can also find the same answer if we do a chain rule differentiation by substituting the value of `y`.

Notebook: [Autograd in pytorch and jax](#)

Detaching and no_grad()

- Both detach() and torch.no_grad() are useful tools for controlling gradient computations in PyTorch.
- detach() is typically used to detach a specific tensor from the computation graph,
- torch.no_grad() is used to temporarily disable gradient computations for a block of code.
- They provide flexibility in managing gradients based on the specific requirements of computation.

```
import torch

# Create a tensor with tracking enabled
x = torch.tensor([3.0], requires_grad=True)

# Perform operations on the tensor
y = x ** 2 + 2 * x + 1

# Stop tracking gradients using detach()
z = y.detach()

# Perform further operations without tracking
with torch.no_grad():
    w = z * 2

print(w)
```

The value of w is tensor([32.]) as no gradient is being calculated.

Advanced autograd operations: Finding the partial derivatives and higher order gradients

```
import torch

x = torch.tensor([2.0], requires_grad=True)
y = torch.tensor([3.0], requires_grad=True)

z = x ** 2 + y ** 3

z.backward(retain_graph=True) # Specify retain_graph=True

print(x.grad) # Partial derivative of z with respect to x
print(y.grad) # Partial derivative of z with respect to y

z.backward()

print(x.grad) # Second-order partial derivative of z with respect to x
print(y.grad) # Second-order partial derivative of z with respect to y
```

The result for the code will be
tensor([4.])
tensor([27.])
tensor([8.])
tensor([54.])

The `retain_graph` parameter in Pytorch's `backward()` function is used to indicate whether the computation graph should be retained after the backward pass.

Question: What happens if we specify `x.grad.zero()` and `y.grad.zero()` after the first backward call?

Advanced autograd operations: Calculating the hessian

- The Hessian matrix is a square matrix that represents the second-order partial derivatives of a scalar-valued function with respect to its variables. It provides information about the curvature and second-order behavior of the function.
- In the context of deep learning and optimization, the Hessian matrix is often used to analyze the behavior of the loss function and to improve the optimization algorithms.
- It helps in understanding the curvature of the loss surface, identifying saddle points, and estimating the step size for optimization methods.

Advanced autograd operations: Calculating the hessian

```
import torch
from torch.autograd.functional import hessian

# Create a tensor with requires_grad=True
x = torch.tensor([[1.0, 2.0, 3.0], [2.5, 3.5, 4.5]], requires_grad=True)

# Define a function that returns a scalar
def scalar_function(x):
    return (x ** 3).sum()

# Compute the Hessian matrix
hessian_matrix = hessian(scalar_function, x)

# Access the Hessian matrix
print(hessian_matrix)
```

```
tensor([[[[ 6.,  0.,  0.],
           [ 0.,  0.,  0.]],

         [[ 0., 12.,  0.],
           [ 0.,  0.,  0.]],

         [[ 0.,  0., 18.],
           [ 0.,  0.,  0.]]],

        [[[ 0.,  0.,  0.],
           [15.,  0.,  0.]],

         [[ 0.,  0.,  0.],
           [ 0., 21.,  0.]],

         [[ 0.,  0.,  0.],
           [ 0.,  0., 27.]]]])
```

Pytorch Eager mode

Eager Mode:

- Eager mode, also known as dynamic computation, is the default execution mode in PyTorch.
- In eager mode, computations are performed immediately as they are encountered in the code, similar to regular Python operations.
- It provides a more interactive and intuitive programming experience, allowing for easier debugging and experimentation.
- Eager mode allows you to easily inspect and manipulate intermediate values during computation.
- It is well-suited for small-scale prototyping and debugging, as well as scenarios where dynamic control flow is needed.

Example of Pytorch Eager mode

```
import torch

# Initializing input tensors
a = torch.tensor(15.0, requires_grad=True)
b = torch.tensor(20.0, requires_grad=True)

# Enable eager mode execution
torch.set_grad_enabled(True)

# Computing the output
c = a * b

# Computing the gradients
c.backward()

# Collecting the output gradient of the output with respect to the input 'a'
derivative_out_a = a.grad

# Collecting the output gradient of the output with respect to the input 'b'
derivative_out_b = b.grad

# Displaying the outputs
print(f'a = {a}')
print(f'b = {b}')
print(f'c = {c}')
print(f'Derivative of c with respect to a = {derivative_out_a}')
print(f'Derivative of c with respect to b = {derivative_out_b}')
```

a = 15.0

b = 20.0

c = 300.0

Derivative of c with respect to a = tensor(20.)

Derivative of c with respect to b = tensor(15.)

- In this code, the eager mode execution is enabled explicitly by setting `torch.set_grad_enabled(True)`. This allows immediate computation and tracking of gradients during the operations.
- `Torch.set_grad_enabled` is true by default

Computational Graph mode using JIT

- JIT (Just-in-Time) is an abbreviation for "Just-in-Time" compilation, which is a technique used in PyTorch to optimize and execute models efficiently.
- Compilation on-demand: JIT enables the compilation of PyTorch models at runtime, just before they are executed.
- This contrasts with ahead-of-time (AOT) compilation, where the entire model is compiled before execution.
- Static graph representation: JIT converts PyTorch models into a static computational graph representation, also known as TorchScript.
- This static graph captures the flow of data and operations in the model and allows for optimizations such as operator fusion and memory optimizations.

Usage of eager mode vs computational graph mode

- Eager mode is used during prototyping and experimentation, and when working with models that involve dynamic control flow, such as recurrent neural networks (RNNs) or models with varying input sizes. It allows use of standard Python control flow statements (e.g., loops, conditionals) to handle dynamic aspects of the model
- Computational graph mode is commonly used for production training, inference and deployment. They are also commonly used in advanced optimization techniques, such as operator fusion, memory optimization, and parallel execution

Example of JIT

```
import torch
import torch.nn as nn

# Define the model as a class
class MultiplyModel(nn.Module):
    def __init__(self):
        super(MultiplyModel, self).__init__()
        self.linear = nn.Linear(1, 1)

    def forward(self, x):
        return self.linear(x)

# Create an instance of the model
model = MultiplyModel()

# Create example inputs
a = torch.tensor([15.0])
b = torch.tensor([20.0])

# Compile the model to a static graph using JIT
model = torch.jit.trace(model, (a * b))

# Execute the model on the inputs
c = model(a * b)

# Display the output
print(f'c = {c}')
```

```
c = tensor([[300.]], grad_fn=<AddmmBackward>)
```

- In the given example `c = tensor([[300.]])`, `grad_fn=<AddmmBackward>`, the tensor `c` represents a 1x1 tensor with a value of 300.
- The `AddmmBackward` operation refers to the backward pass of the matrix multiplication with addition operation (`torch.addmm`).
- The `trace` function executes the model with the provided input(s), captures the control flow and dependencies of the model, creating a computational graph.

Summary

- In this topic we understood Autograd.
- We delved into automatic differentiation, and manually calculated gradients for functions.
- We also got an understanding of doing advanced operations like Hessian and partial differentiation with Autograd.
- The final topics covered the eager mode and just-in-time mode used in deep learning frameworks.

Practice code

Code used in the notebook: [Slide code](#)

Loading and analysing the MNIST dataset: [MNIST dataloader](#)

A two layer neural network: [Neural Network](#)

Thank you